GIT PUSH THE BEATS

Melody Dive
Software Architecture Document

Version 1.2

Revision History

Date	Version	Description	Author
23/11/2025	1.0	Define software architecture	Ngo Tran Quang Dat Nguyen Hoang Anh Khoa
01/12/2025	1.1	Add details information about the Package: view, routers, service, models	Ngo Tran Quang Dat Nguyen Hoang Dang Nguyen Hoang Anh Khoa Thai Minh Huy Huynh Gia Bao
17/12/2025	1.2	Add detail information about Deployment, Implementation View section	Thai Minh Huy Huynh Gia Bao

Table of Contents

Software Architecture Document

1. Introduction

The introduction of the Software Architecture Document provides an overview of the entire Software Architecture Document. It includes the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the Software Architecture Document.

Purpose: The purpose of this document is to provide a comprehensive architectural overview of the Melody Dive system. It is intended to capture and convey the significant architectural decisions which have been made on the system.

Scope: The scope of this document is limited to the architectural design of the Melody Dive system. It includes:

- The architectural goals and constraints that shape the system's design.
- The decomposition of the system into major subsystems and components, specifically focusing on the interaction between the Frontend and the Backend (Logical View).
- The significant design patterns selected and the justification for their use.
- The high-level deployment strategy and code organization (Deployment and Implementation Views).

Acronyms, abbreviations:

- SAD: Software Architecture Document.
- API: Application Programming Interface.
- UI: User Interface.
- FE: Frontend (Client-side).
- BE: Backend (Server-side).

References:

- Tuan Thanh HO. (2020, November 28). PA3 PA4 [Video]. YouTube. Retrieved November 23, 2025, from https://www.youtube.com/watch?v=pcM9xQiUt5g

Overview: This document is organized as follows:

- *Section 1* provides an overview of the Software Architecture Document.
- *Section 2* describes the architectural goals and constraints.
- *Section 3* provides references to the Use-Case Model.
- *Section 4* details the Logical View, describing the major software components and their relationships.
- *Section 5* describes the Deployment View (Hardware mapping).
- *Section 6* describes the Implementation View (Code structure).

2. Architectural Goals and Constraints

2.1 Architectural Goals

- **Availability:**
  - The system must remain available 24/7, allowing users to stream music at any time.
- **Usability:**
  - New users should be able to search and play at least one song during their first usage session, within no more than 5 interactions.
  - After 30 minutes of usage, users should be able to utilize at least 90% of core functionalities.
- **Performance:**

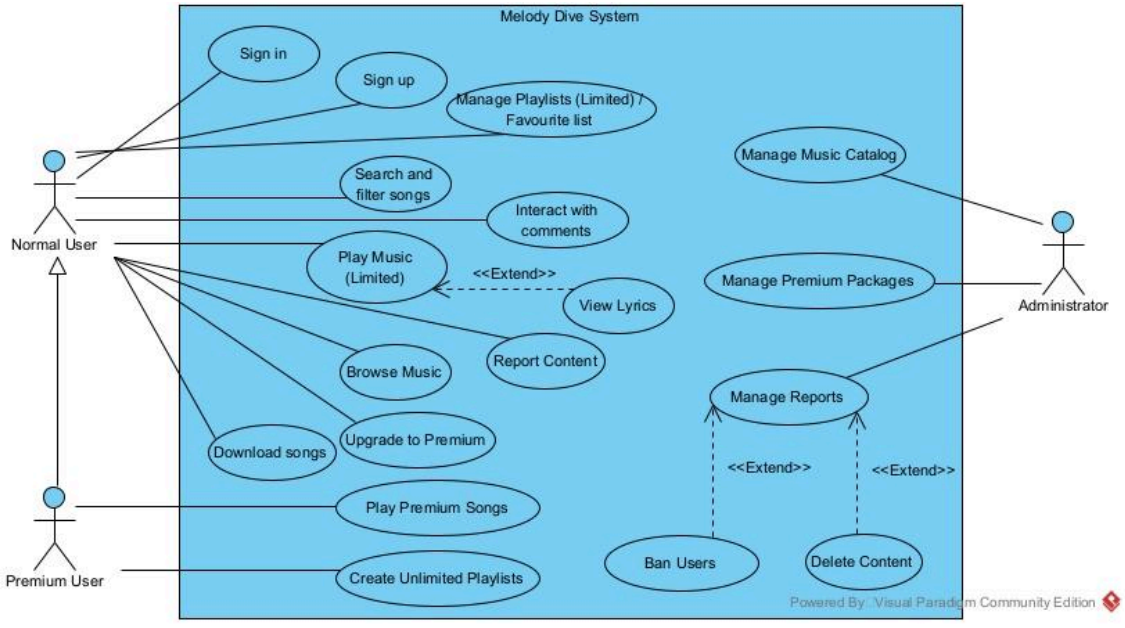
- Initial playback time (from clicking “Play” to hearing audio) must not exceed 2 seconds for 95% of playback requests.
- UI controls such as play/pause/next/previous must respond within ≤ 500 ms
- Search results must be returned within ≤ 1 second for 95% of queries.

- **Reliability:**
  - The probability of system inaccessibility during peak hours must be < 0.5%.
  - The error rate for accessing songs, playlists must be < 1% of total system requests.
- **Robustness:**
  - System recovery time after a failure must be < 10 minutes.
  - User data must not be lost after a failure, with a data corruption rate ≤ 0.1%.

2.2 Architectural Constraints

- Frontend must be developed using ReactJS and standard web technologies including HTML, CSS, and JavaScript.
- Backend must be implemented using FastAPI written in Python.
- The development environment is constrained to Visual Studio Code, following the team’s workflow and extensions.
- The application must operate as a web-based system (desktop & mobile browsers), not native mobile app.
- Must follow the team’s development toolchain, including GitHub for version control and issue tracking.
- Must comply with security constraints, including safe authentication for user accounts and premium access control.
- Must ensure maintainability by reusing existing UI components and backend modules developed in earlier milestones.

3. Use-Case Model



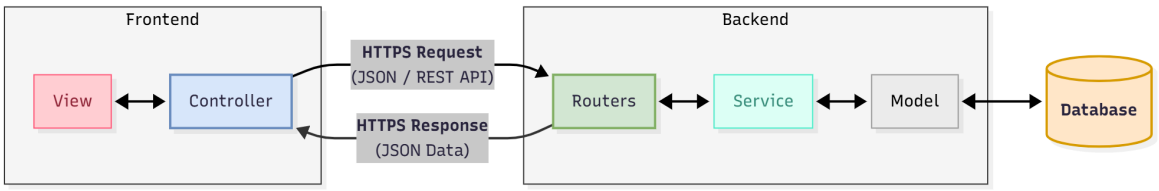
4. Logical View

The software architecture of Melody Dive follows a Client-Server architectural style. The system is distributed across two main computing environments: the client's web browser and the centralized application server.

To ensure separation of concerns and maintainability, the system is decomposed into three high-level components (containers):

- Frontend Single-Page Application (SPA): Handles user interaction and presentation.
- Backend API Application: Handles business logic and security.
- Database System: Handles data persistence.

The following diagram illustrates the high-level architecture and the communication pathways between these components.

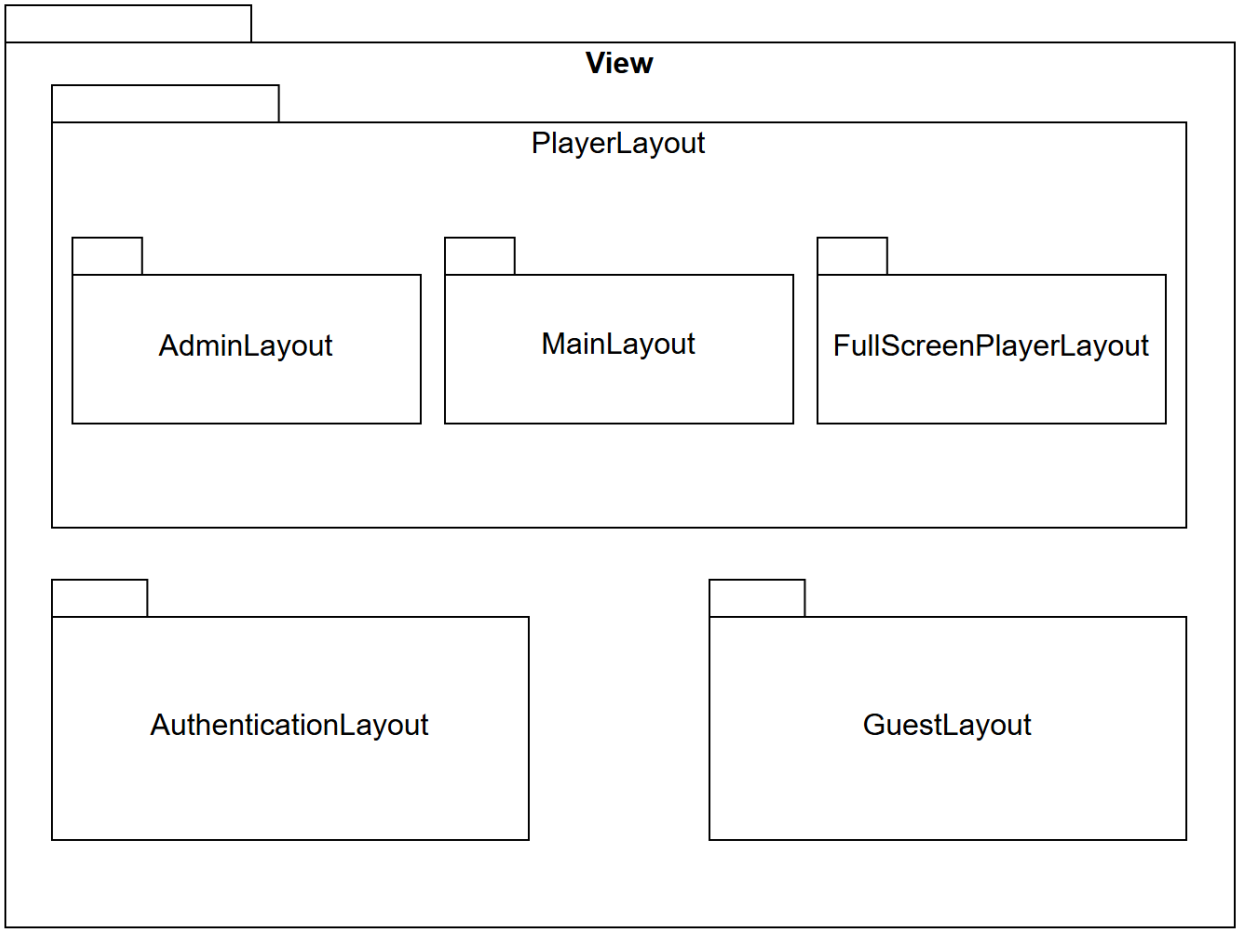


Descriptions:

- Frontend (ReactJS)
  - View: Renders the graphical user interface, displays data to the user, and captures user interactions (such as clicking "Play" or typing a search query).
  - Controller: Handles events triggered by the View, manages the local application state (e.g., current song, user session), constructs HTTPS requests to send to the backend, and processes the JSON responses to update the UI.
- Backend (FastAPI)
  - Routers: The API controller layer implemented using FastAPI Routers. It serves as the entry point for the server. It receives incoming HTTPS requests, validates the request data using Pydantic schemas, and routes the request to the appropriate business logic function in the Service layer.
  - Service: The business logic layer containing the core rules and algorithms of Melody Dive. It executes complex operations such as processing AI semantic search queries, calculating trending songs, hashing passwords for authentication, and coordinating data transactions.
  - Model: Abstraction of the database structure. It translates high-level Python objects into low-level SQL queries to perform CRUD (Create, Read, Update, Delete) operations on the database.
- Database: The external relational database system (PostgreSQL).

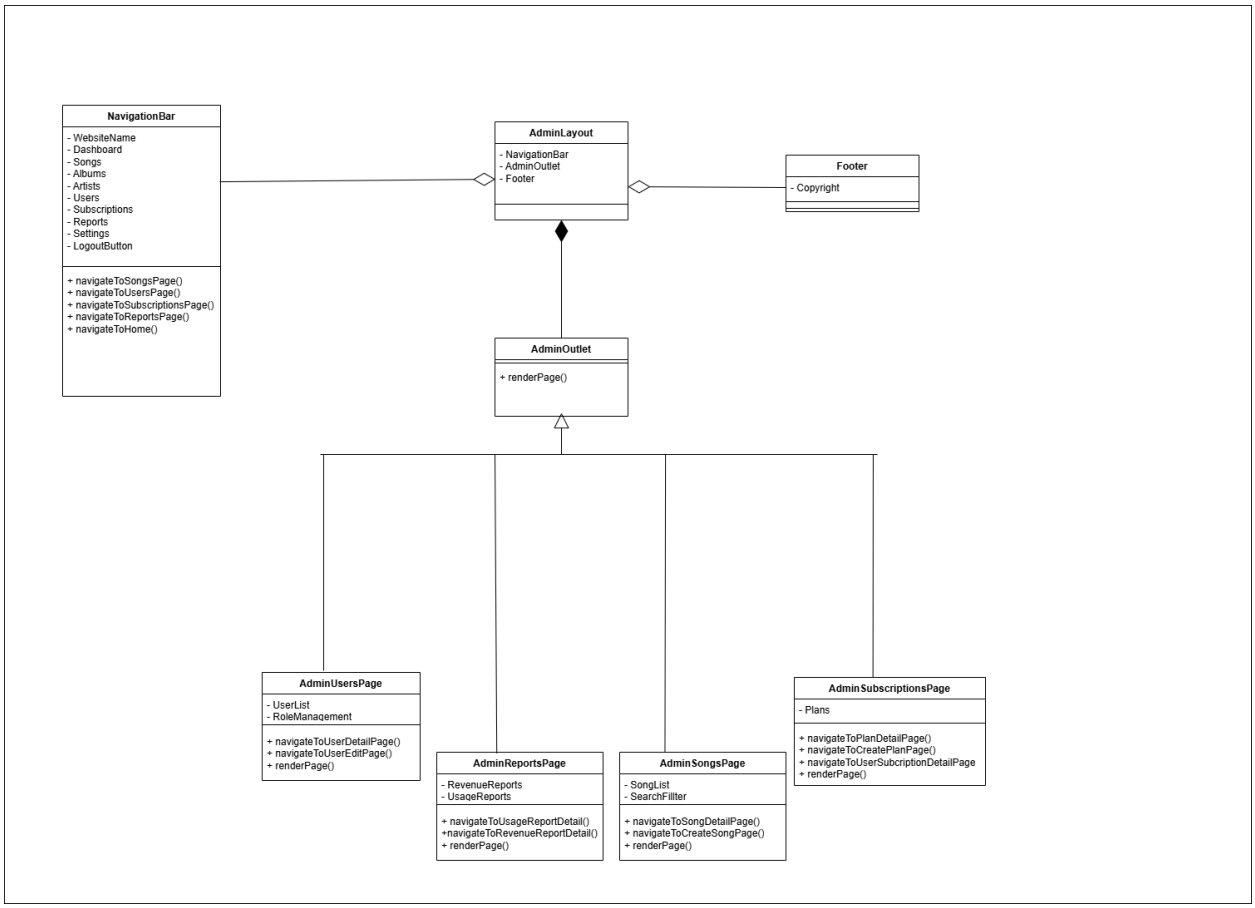
4.1 Package: View

- **View Package Diagram:**



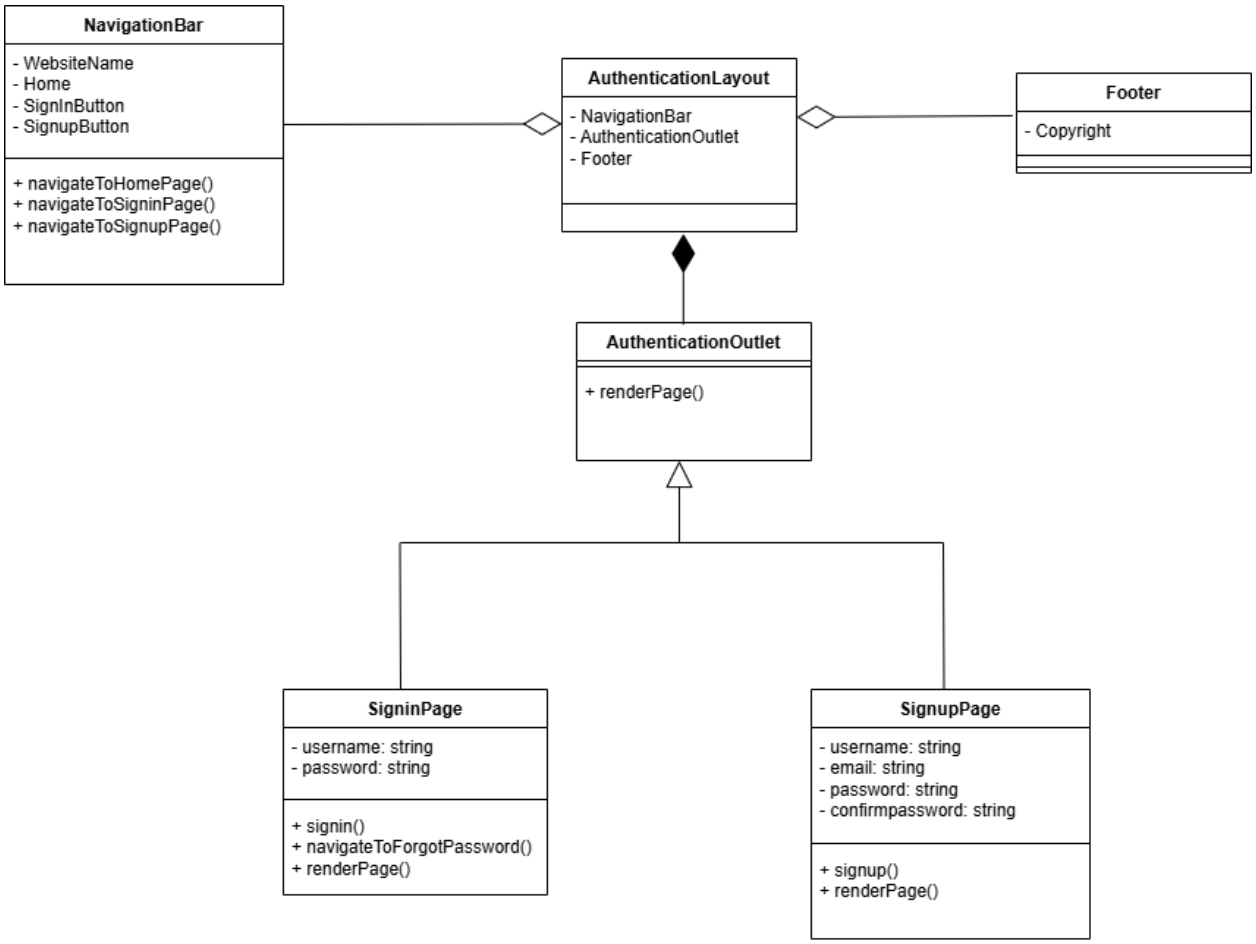
Description: Package View includes 6 packages that represent different layouts, and each layout can contain React components along with an outlet to render either a page or another layout. Splitting layouts and using an outlet is a powerful approach that allows an SPA to take advantage of nested routes, where smaller routes belonging to a specific layout are grouped under that layout. This technique is also important in music-player web apps, helping the system define which page transitions or layout switches should keep the mini player bar running without any interruption.

4.1.1 AdminLayout



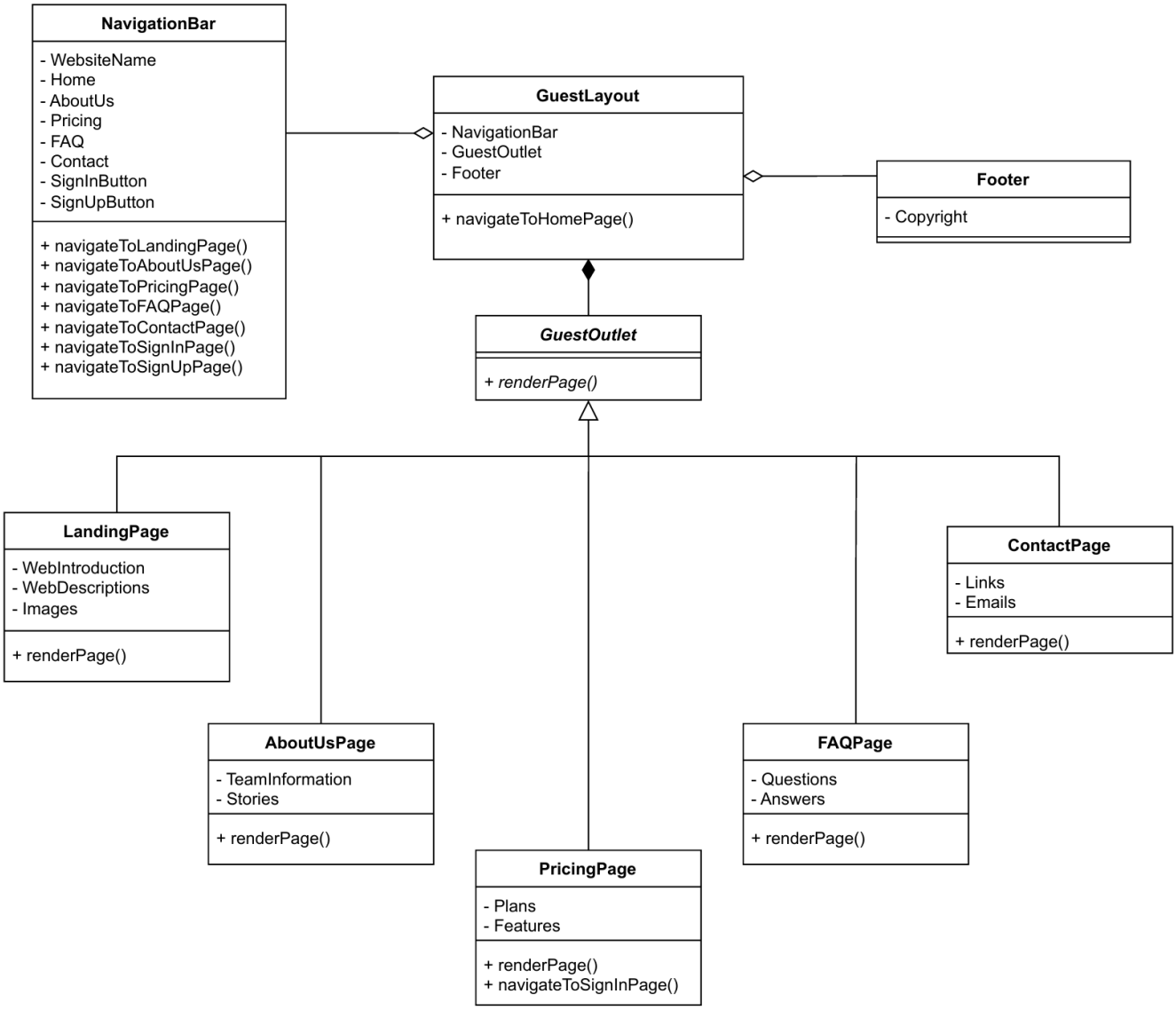
Description: This layout represents the administrative interface of the website. It organizes navigation and rendering of pages for managing users, reports, songs, and subscriptions, each providing related detailed navigation. actions.

4.1.2 AuthenticationLayout



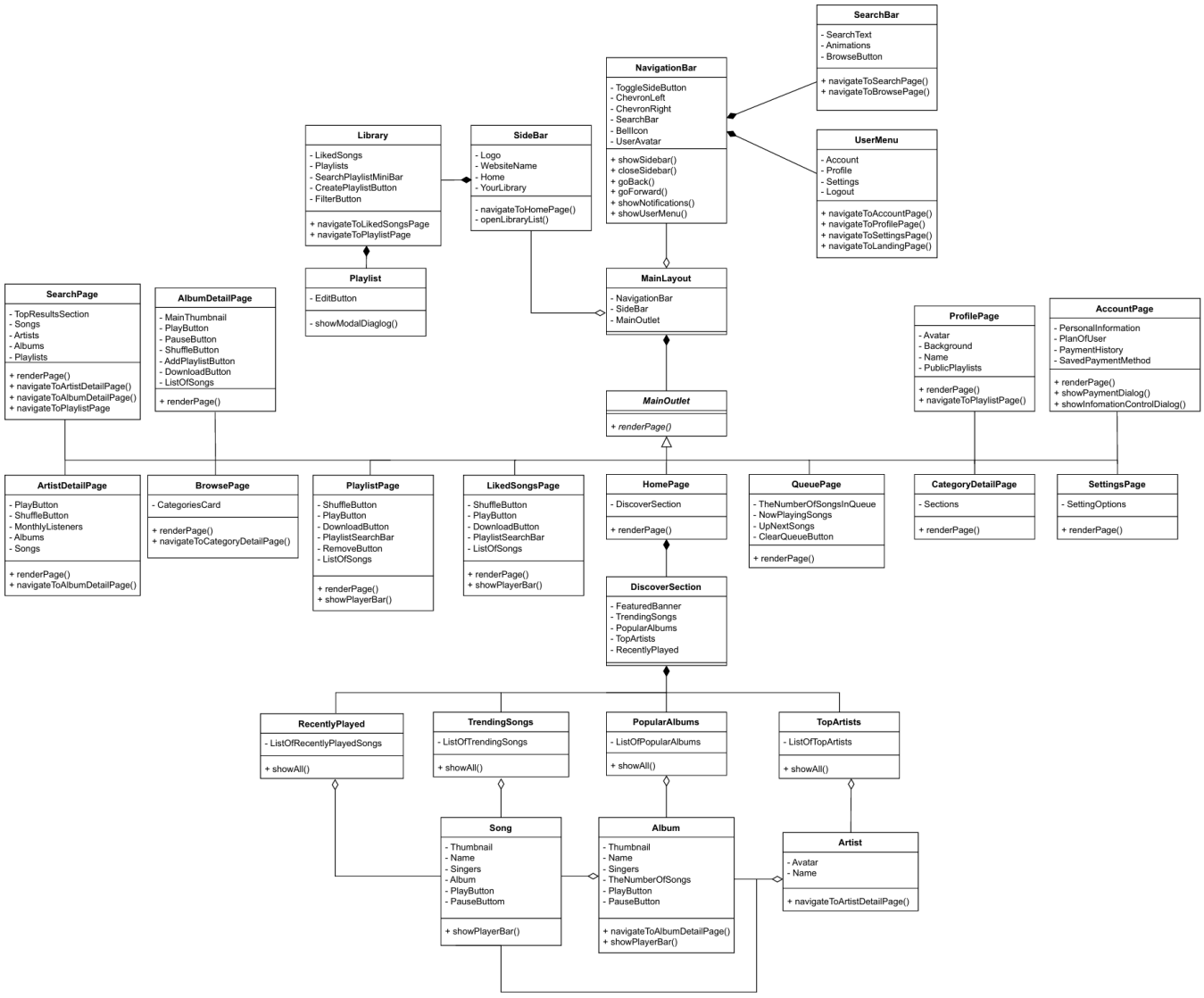
Description: This layout represents the authentication interface of the website. It coordinates the navigation bar, authentication layout, and outlet to render the Signin and Signup pages, which provide operations for submitting user credentials and navigating between authentication flows (e.g., sign in, sign up, forgot password).

4.1.3 GuestLayout



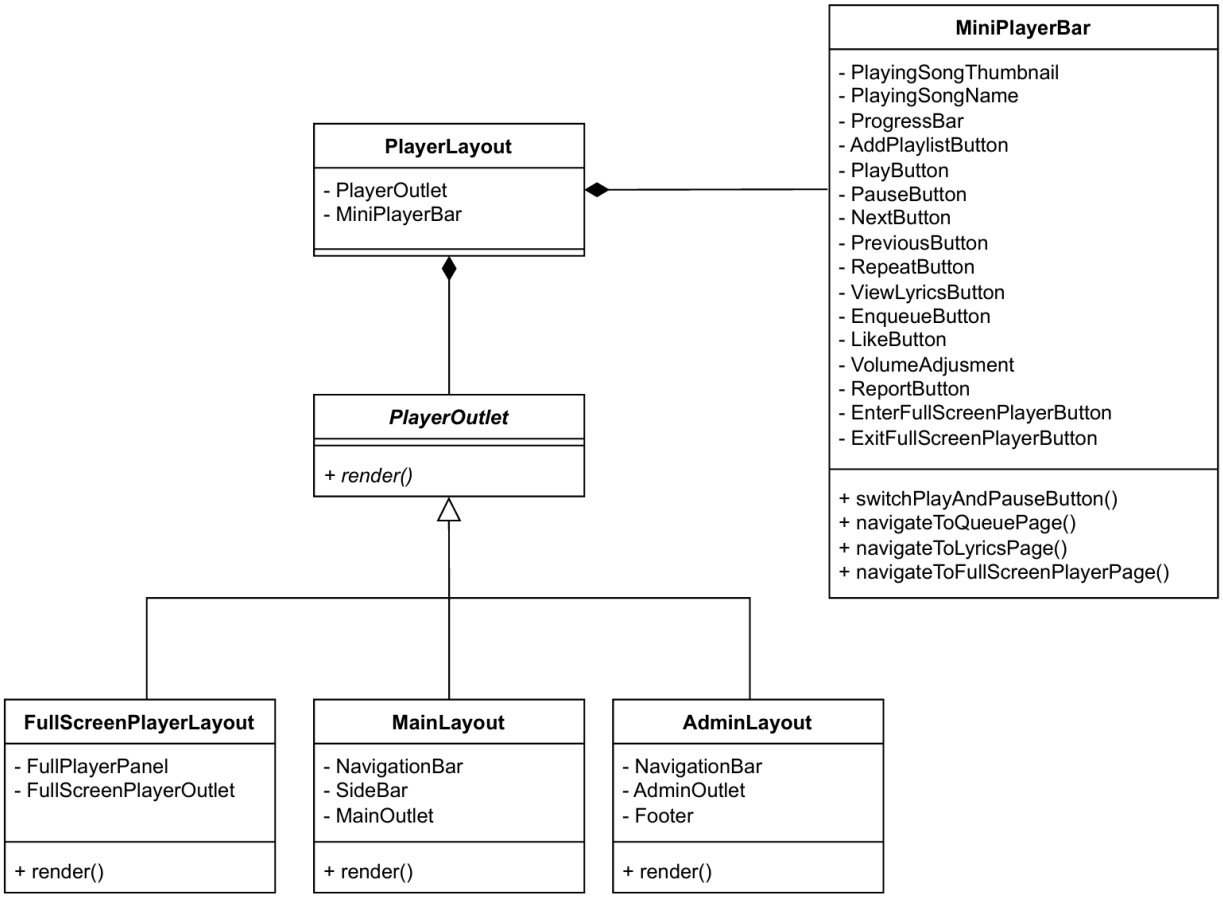
Description: GuestLayout mostly consists of static pages that introduce the product. These are also the first pages users see when they visit the site for the first time, helping them understand what our platform does, what features it provides, common questions, and the available service plans if they want to upgrade their experience.

4.1.4 MainLayout



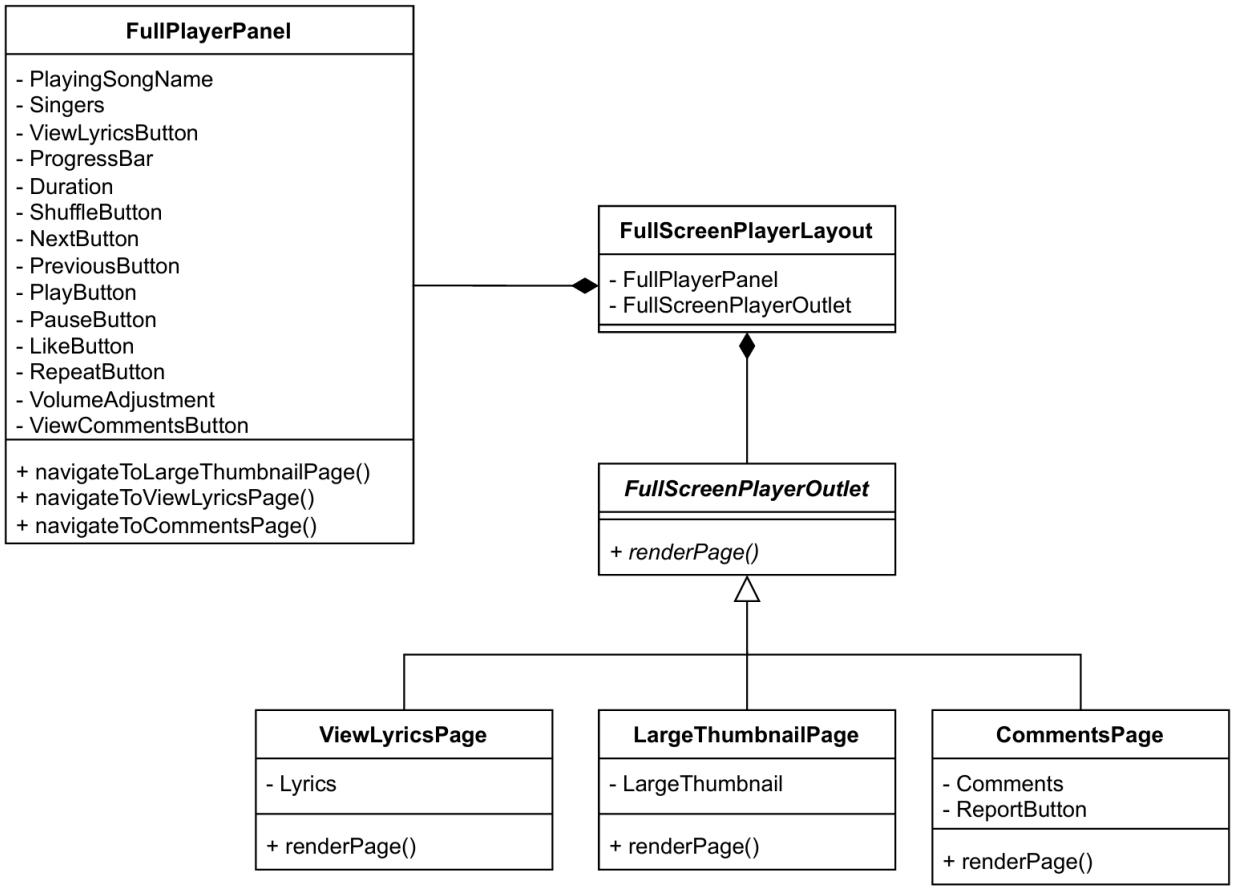
Description: This interface is the main UI that powers the entire music listening experience for users after they decide to become a member of our platform. The overall layout includes a top navbar and a left sidebar that help users navigate smoothly between pages, along with a search bar for songs, playlists, artists, and even account management sections like profile and upgrading or downgrading their premium plan. Additionally, it also features an eye-catching home page that leaves a strong first impression the moment users log in, with a feature banner and trending songs, albums, and artists.

4.1.5 PlayerLayout



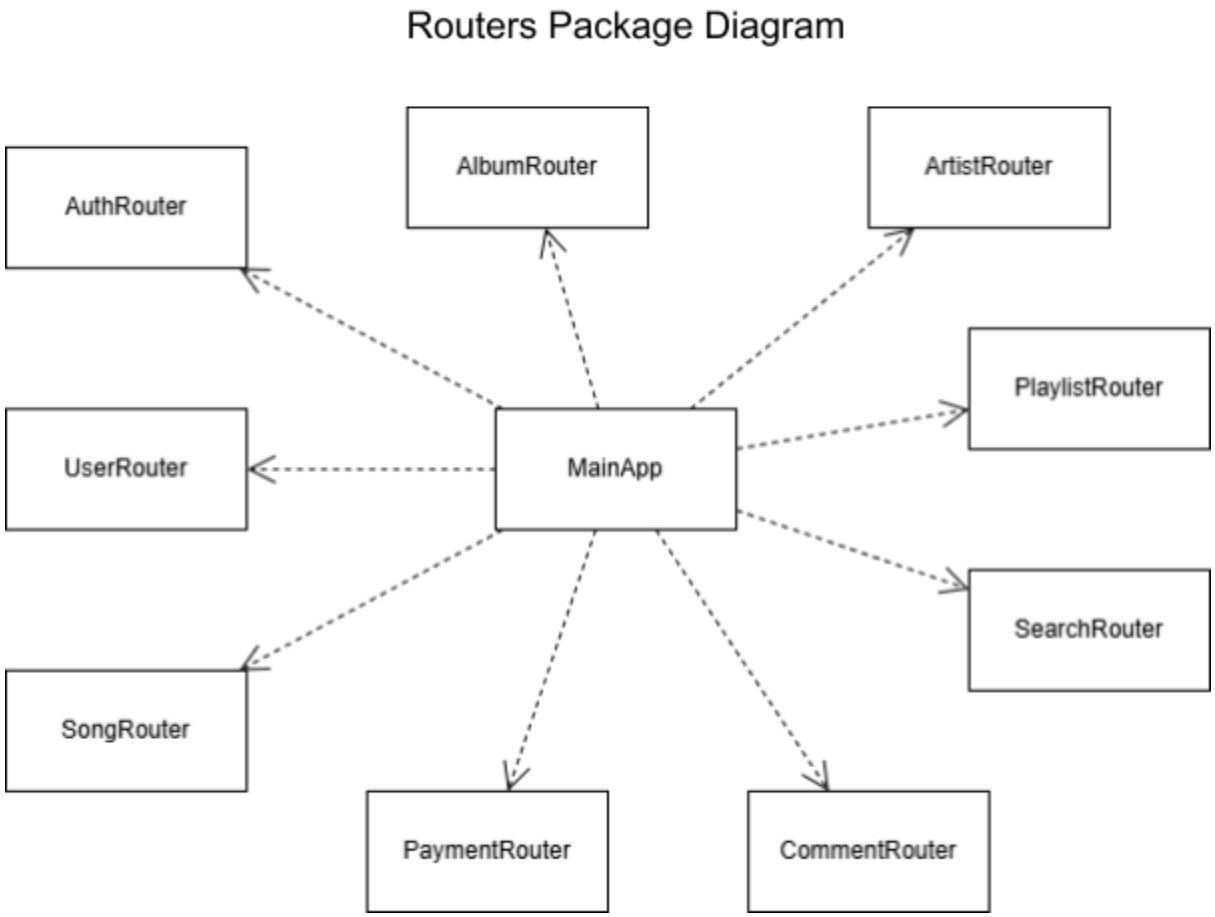
Description: PlayerLayout is a bit different from the other layouts, it doesn't render any actual page but instead renders other layouts, because it's not truly a page layout on its own. Its main purpose is to host a persistent mini player that stays uninterrupted as users move between pages inside the layouts it wraps. In terms of UI, the mini player displays all relevant information about the current track and lets users interact with convenient features directly on it, such as seeking, pausing, playing, liking, shuffling, viewing lyrics, and more.

4.1.6 FullScreenPlayerLayout



Description: The group of pages in this layout enhances the user experience when they want the music interface to expand to full size, turning their device into a music background while they work or study. In this section, users can also conveniently view lyrics and the comment section of the currently playing track, and if they prefer not to be distracted, they can easily switch back to the thumbnail view.

4.2 Package: Routers

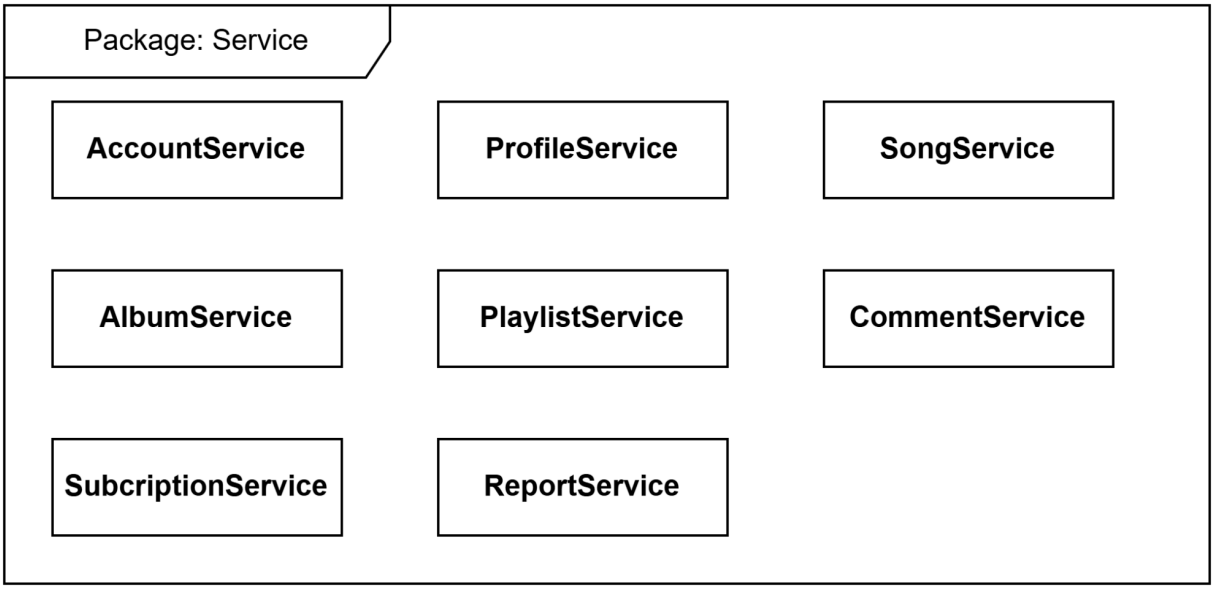


Description: The Routers package serves as the primary entry point for all client requests, exposing a RESTful API interface to the Frontend. The Routers Package Diagram illustrates the modular architecture of the FastAPI backend. The central MainApp component acts as the application entry point, aggregating distinct functional Routers (e.g., AuthRouter, SongRouter) via dependency injection. This structure ensures a clean separation of concerns by isolating API endpoints based on their domain.

The key sub-components include:

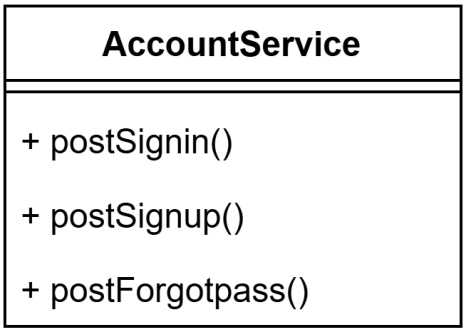
- Authentication & users (AuthRouter, UserRouter): Manages the security context. It handles user registration, login (JWT issuance), password management, and administrative actions (banning users).
- Music catalog (SongRouter, AlbumRouter, ArtistRouter): Provides endpoints for retrieving music metadata, lyrics, and streaming URLs. It allows the frontend to navigate the hierarchy of Artist → Album → Song.
- Discovery (SearchRouter): A specialized controller that handles search queries.
- User interaction (PlaylistRouter, CommentRouter): Manages user-generated content, including creating custom playlists, "liking" songs, posting comments, and reporting toxic content.
- Subscription (PaymentRouter): Handles the business logic for upgrading users to Premium status.

4.3 Package: Service



Description: Encapsulate the application's business logic. This layer receives validated data from the Routers, interacts with the **Models** to retrieve or update data in the database, and performs necessary calculations or logic checks before returning the result. It ensures that business rules are applied consistently.

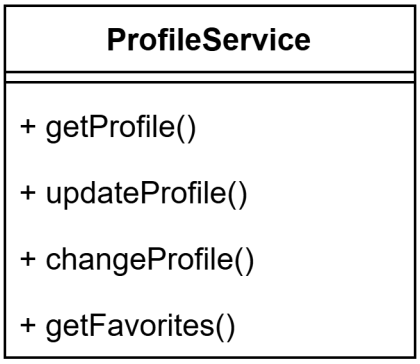
4.3.1 AccountService



Purpose: Handle user authentication and account recovery operations:
Functions:

- PostSignin(): Checks credentials (username/email and password) and authenticates the user, returning a JWT if correct.
- PostSignup(): Registers a new user account in the system if the provided email and username are unique.
- PostForget_Pass(): Initiates the account recovery process by sending a password reset email or OTP to the user.

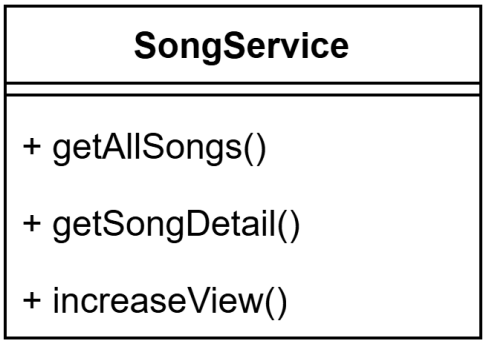
4.3.2 ProfileService



Purpose: Manage authenticated user's personal information and preferences
Functions:

- getProfile(): Retrieves the current user's personal details, including full name, avatar, and birth date.
- updateProfile(): Updates the user's general information such as display name, bio, or nationality.
- changeProfile(): Handles sensitive updates, such as changing the password or updating the avatar image.
- getFavorites(): Retrieves the list of songs that the user has marked as favorites.

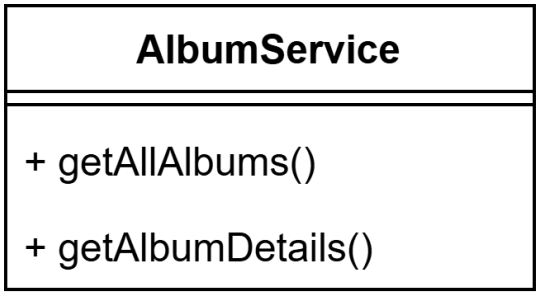
4.3.3 SongService



Purpose: Manage the retrieval of music tracks and track engagement metrics
Functions:

- getAllSongs(): Retrieves a list of songs available in the system, supporting filters by genre, year, or artist.
- getSongDetail(): Fetches specific metadata for a song, including audio file links, lyrics, and composer information.
- increaseViews(): Increments the play count of a specific song when a user streams it.

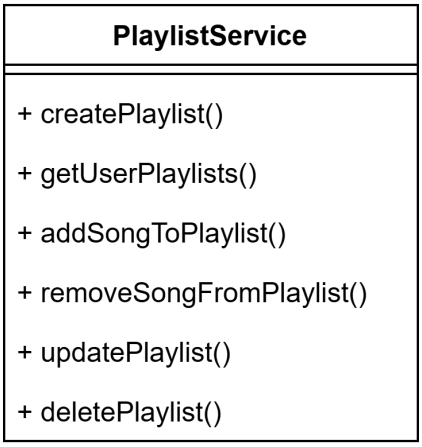
4.3.4 AlbumService



Purpose: Handle the retrieval of album collections and their associated content
Functions:

- getAllAlbums(): Retrieves a list of all available albums, often sorted by release date or popularity.
- getAlbumDetails(): Fetches detailed information about a specific album, including its cover image and the list of songs it contains.

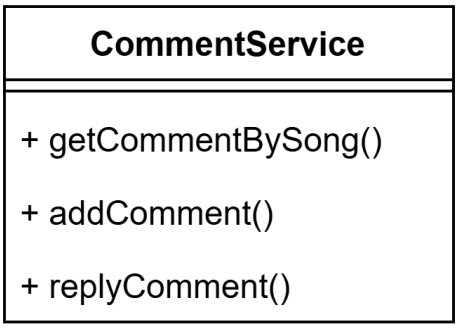
4.3.5 PlaylistService



Purpose: Enable users to manage personal music collections
Functions:

- createPlaylist():Creates a new empty playlist for the user with a specified name.
- getUserPlaylists(): Retrieves a list of all playlists created by the current user, differentiating between normal and premium playlists.
- addSongToPlayList(): Adds a specific song to a selected user playlist.
- removeSongFromPlayList(): Removes a specific song from a selected playlist.
- updatePlaylist(): Modifies playlist metadata, such as renaming the playlist or changing its cover image.
- deletePlaylist(): Permanently removes a playlist and its associations from the user's account.

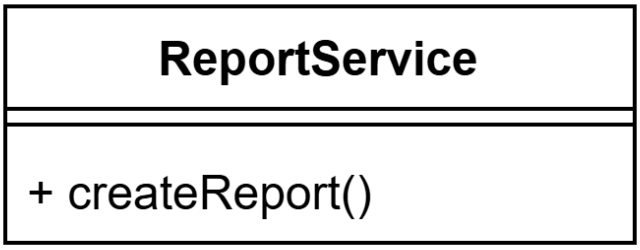
4.3.6 CommentService



Purpose: Manage social interactions and user feedback on songs.
Functions:

- getCommentBySong(): Retrieves all comments associated with a specific song, ordered by time.
- addComment(): Posts a new comment from the user onto a specific song.
- replyComment(): Posts a reply to an existing comment, creating a nested conversation thread.

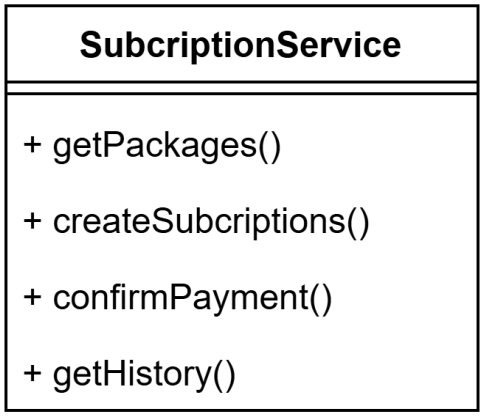
4.3.7 ReportService



Purpose: Handle content moderation and user safety reports
Functions:

- createReport(): Submits a new report against a user, song, or comment for review by administrators.

4.3.8 SubscriptionsService



Purpose: Handle premium package selection and payment processing
Functions:

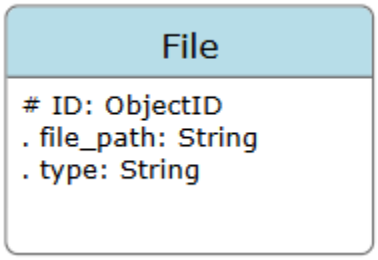
- getPackages(): Retrieves the list of available premium subscription plans and their prices.
- createSubcriptions(): Initiates a new subscription request and generates a pending transaction record.
- confirmPayment(): Validates the user's payment and updates the subscription date.
- getHistory(): Retrieves the user's past transaction history.

4.4 Package: Models

The system consists of 11 main models: **File, Account, Person, Song, Album, Playlist, UserFavorites, Comment, Report, PremiumPackage, Subscriptions.**

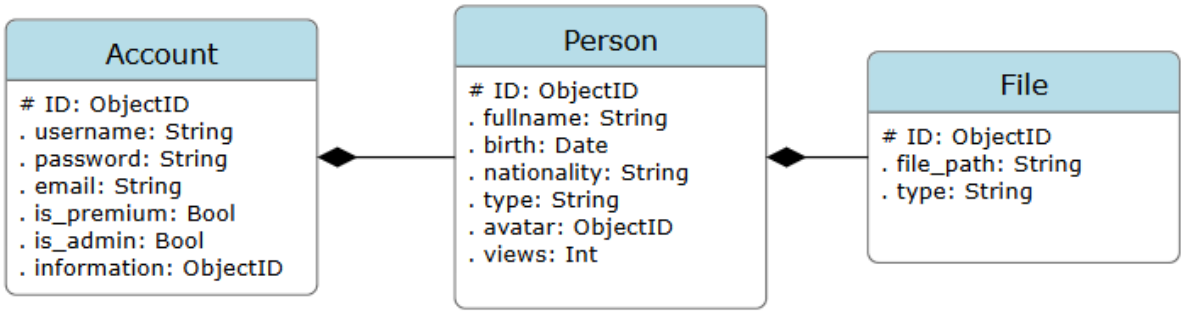
Besides, there are some supporting models with the purpose of connecting main models: **SongArtist, SongComposer, AlbumArtist, AlbumSong, UserPlaylist, PlaylistSong.**

4.4.1 File



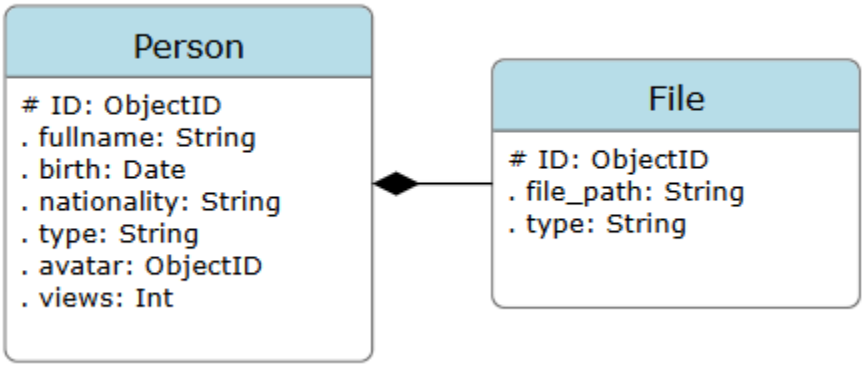
- This model stores the files' path on cloud (*file_path)*, as well as its type (*type*) – image, audio, or text.

4.4.2 Account



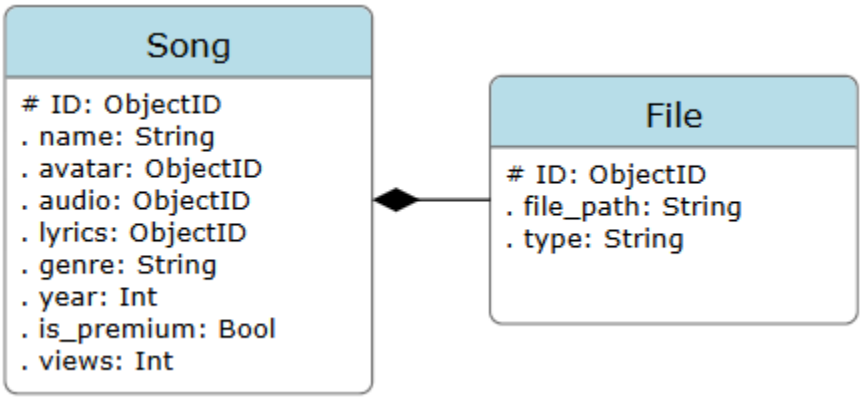
- This model stores users' and admins' accounts.
- The attributes in details:
  + ID: The model's primary key.
  + username: Account's username.
  + password: Account's password, for authentication.
  + email: Account's email.
  + is_premium: Identify whether the account is normal or premium.
  + is_admin: Identify whether the account is a user account or an admin account.
  + information: This attribute is a foreign key of **Person(ID)**. The **Person** model stores personal information of a person (users/admins in this case), which will be mentioned in details below.

4.4.3 Person

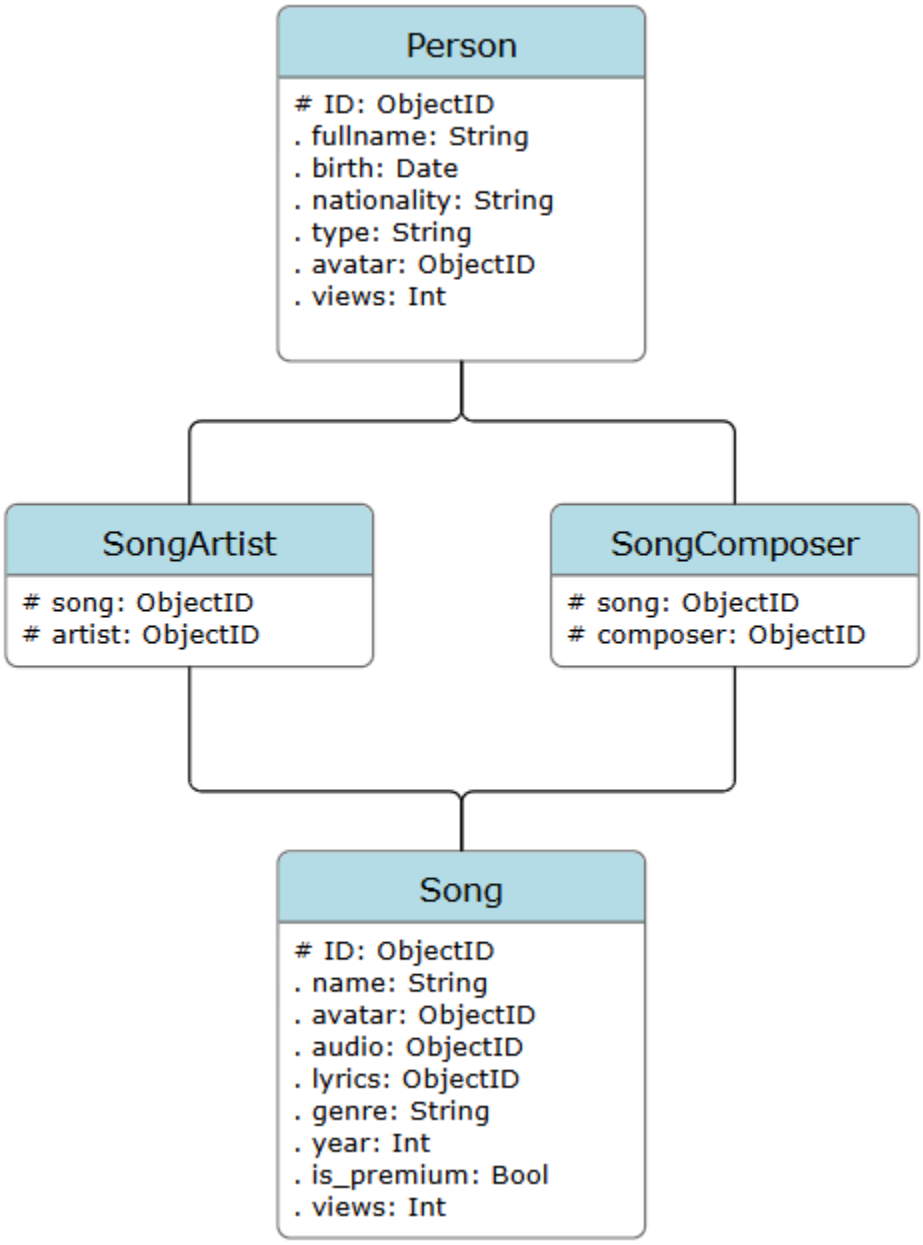


- This model stores personal information of a person ("person" can be a user, admin, song artist, or song composer).
- Attributes:
  + ID: The model's primary key.
  + fullname, birth, nationality: are the person's fullname, birthday, and nationality, respectively.
  + type: type of person (one of the 4 types: user, admin, composer, artist).
  + avatar: this attribute is a foreign key of **File(ID)**.
  + views: (for artists and composers) This attribute is updated automatically when users listen to artist's/composer's songs, and is resetted after a month. This attribute is to determine which artists/composers are popular. (for users/admins, this attribute = None)

4.4.4 Song

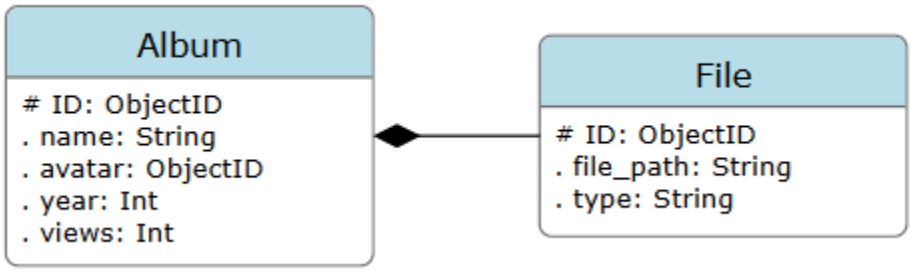


- This model stores songs' metadata and its fundamental files, including its avatar file, audio file and lyrics file.
- Attributes:
    + ID: Model's primary key.
    + name: song's name.
    + avatar, audio, lyrics: are foreign keys of **File(ID)**, corresponding to the song's avatar file, audio file, and lyrics file.
    + genre: the song's genre. (eg: Rap, Pop, etc)
    + year: the song's published year.
    + is_premium: identify if the song is only available for premium users.
    + views: the number of the song's views. (this attribute is resetted each month)
- There are 2 supporting models to model **Song:** SongArtist and SongComposer

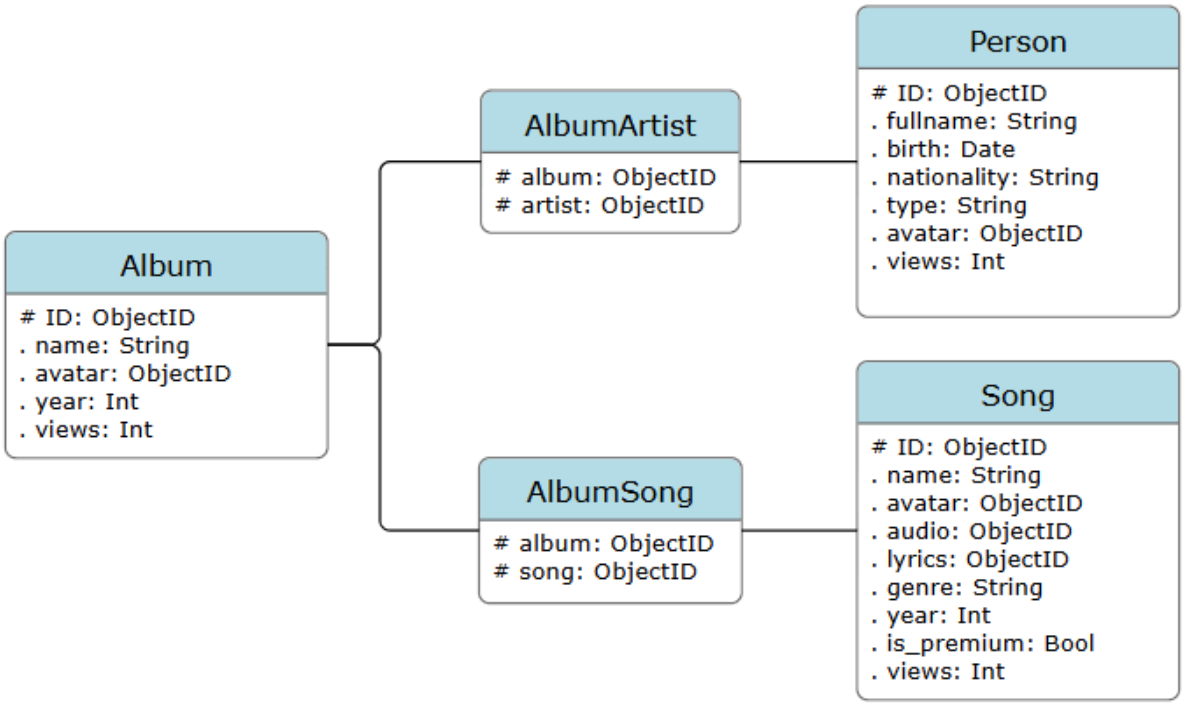


+ These 2 models connect **Person** and **Song** models, to store the information about the artists and composers of songs.

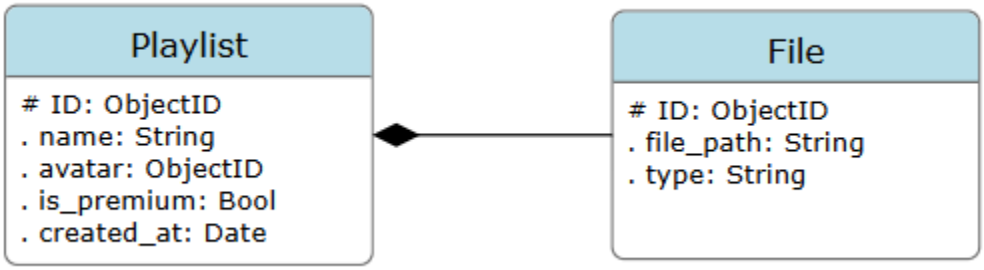
4.4.5 Album



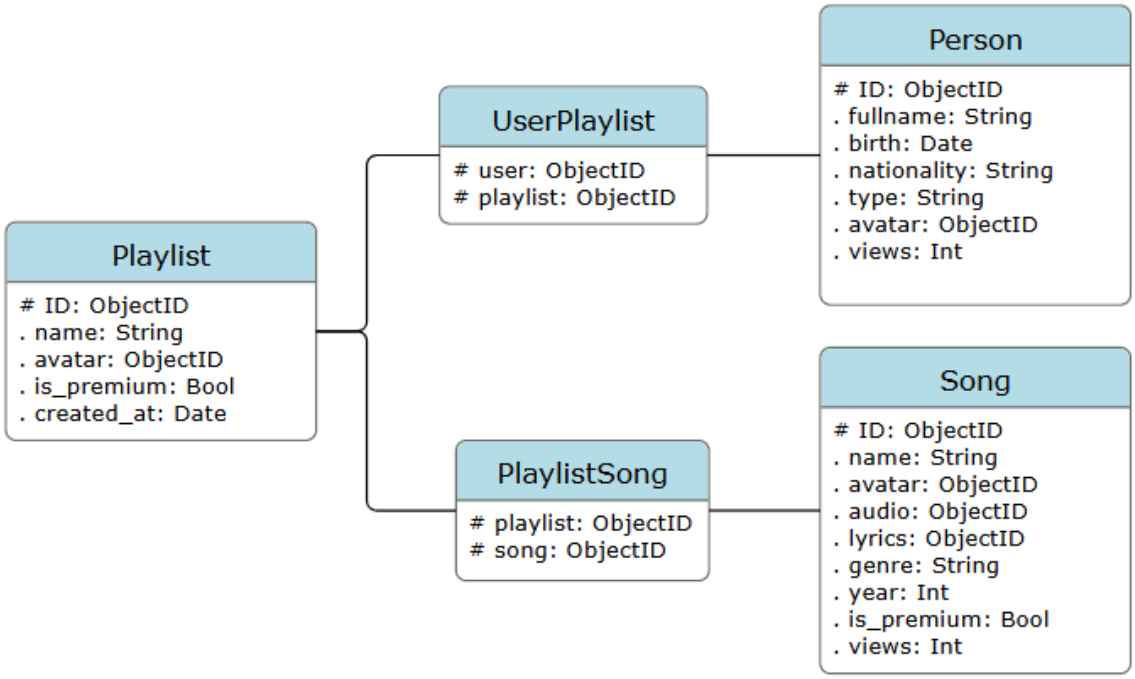
- This model stores albums' information.
- Attributes:
  - + ID: the model's primary key.
  - + name: the album's name.
  - + avatar: the foreign key of **File(ID)**, stores the album's avatar.
  - + year: published year.
  - + views: the quantity of album's views, equals to total number of views of its songs, and is updated automatically.
- Moreover, this **Album** model has 2 supporting models named **AlbumSong** and **AlbumArtist** to store its songs and artists.



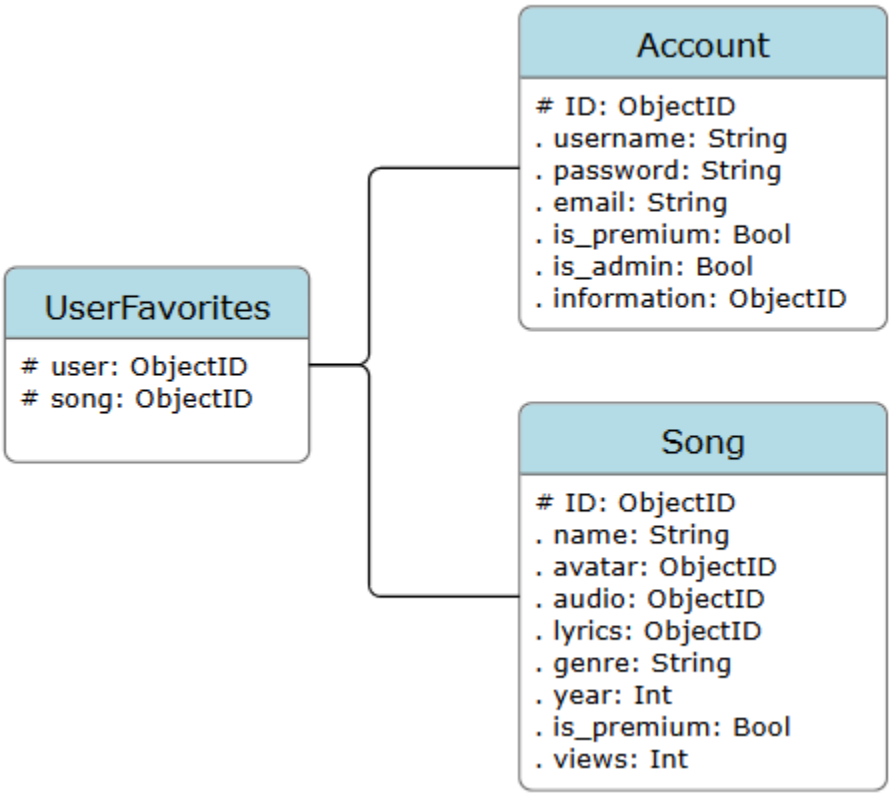
4.4.6 Playlist



- This model stores users' playlists' information.
- Attributes:
  - ID: the model's primary key.
  - name: the playlist's name.
  - avatar: the playlist's avatar. This attribute is a foreign key of **File(ID)**.
  - is_premium: identify whether the playlist is a normal album or premium playlist.
  - created_at: the date when the playlist was created.
- This model has 2 linked models to help store the playlist's song and the user to which it belongs: **PlaylistSong** and **UserPlaylist**.

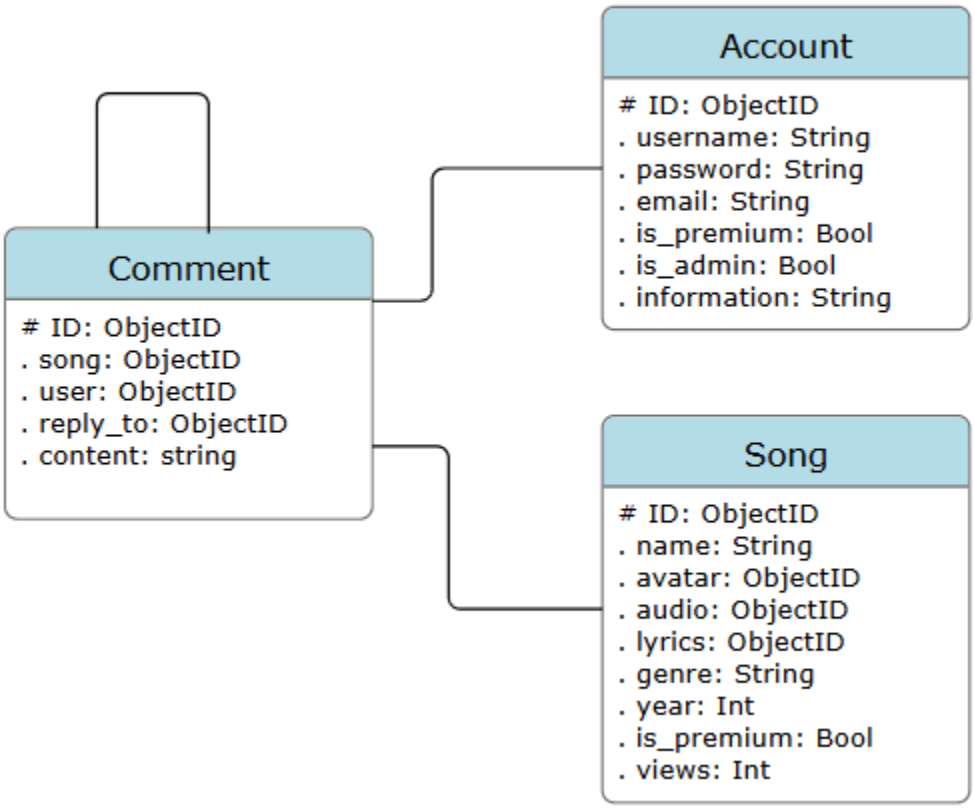


4.4.7 UserFavorites



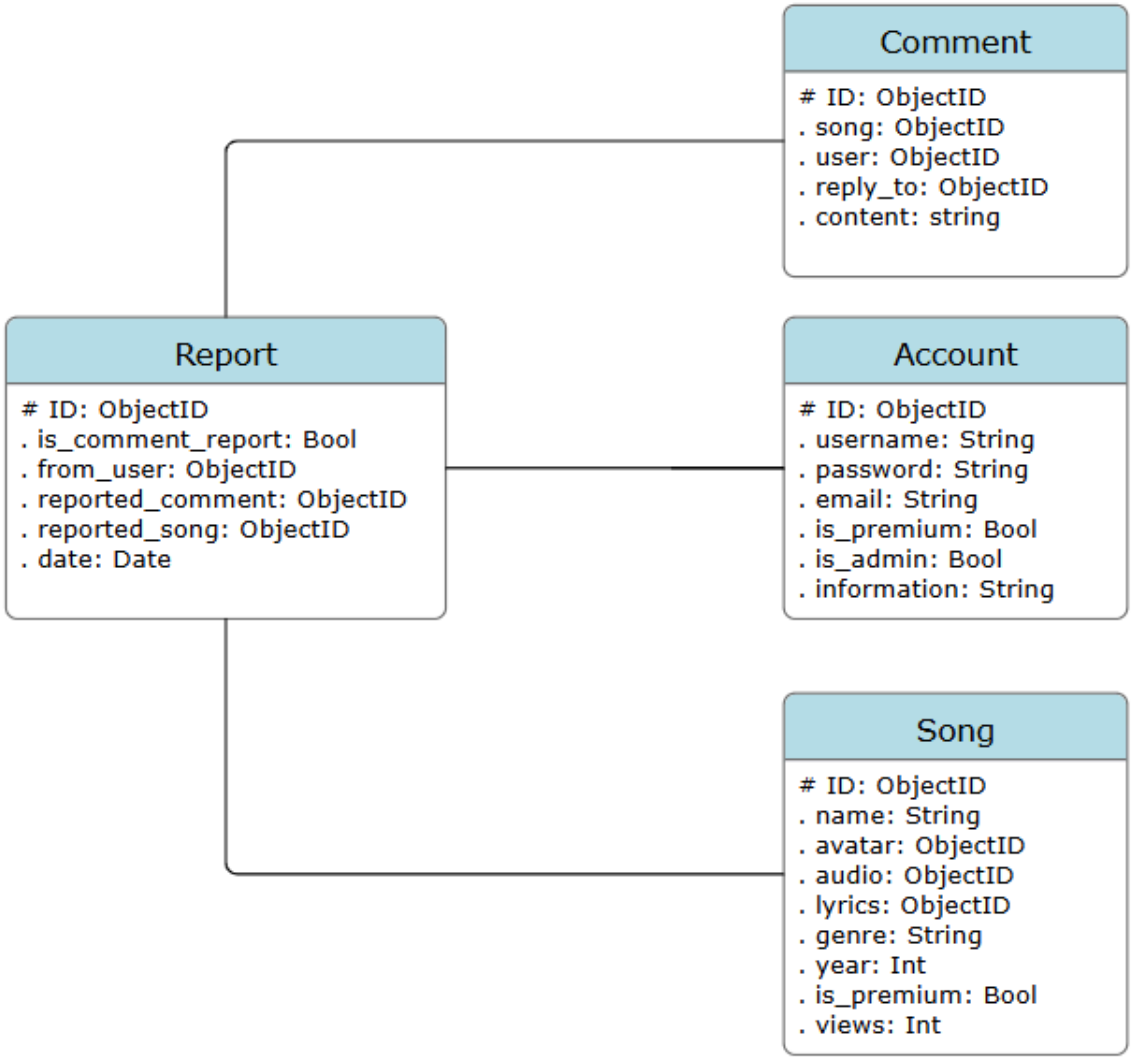
- This model stores users' favorite songs.
- Attributes:
  - + user: the user's account. This attribute is the foreign key of **Account(ID)**.
  - + song: the song. This attribute is the foreign key of **Song(ID).**

4.4.8 Comment



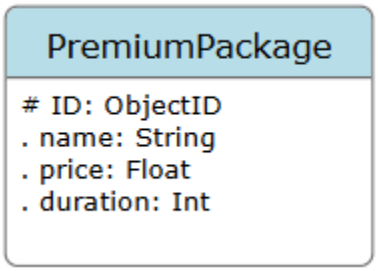
- This **Comment** model stores all user comments of the system.
- Attributes:
  + ID: the model's primary key.
  + song: the song in which the comment is posted. This attribute is the foreign key of **Song(ID)**.
  + user: the account which posted the comment. This attribute is the foreign key of **Account(ID)**.
  + reply_to: the comment to which this comment replies. This attribute is the foreign key of **Comment(ID)**.

4.4.9 Report



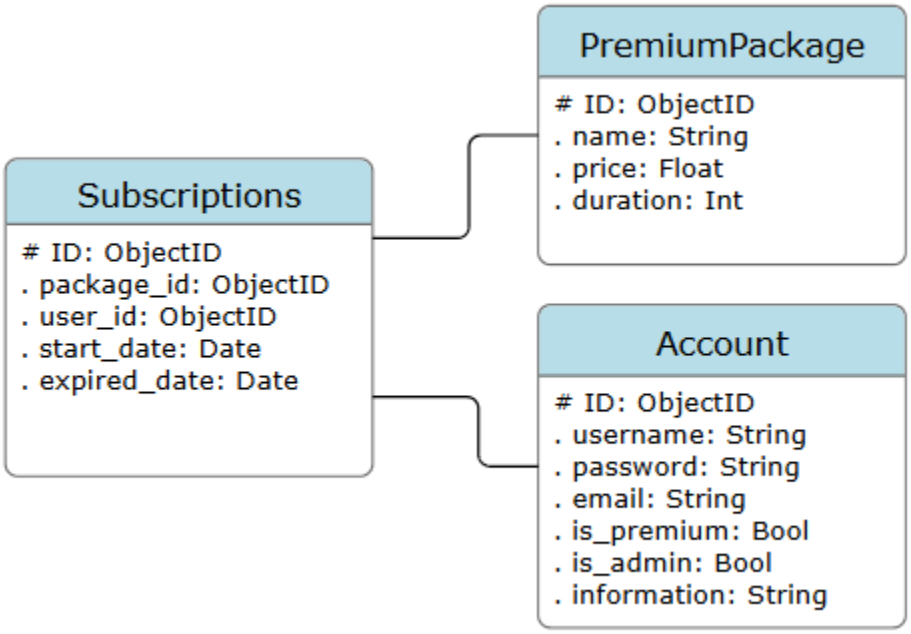
- This **Report** model stores all reports that users report to admins.
- Attributes:
  - + ID: the model's primary key.
  - + is_comment_report: identify whether the report is a song-report or comment-report.
  - + from_user: the user who sends the report. This attribute is the foreign key of **Account(ID)**.
  - + reported_comment: the reported comment (= None if this is a song-report). This attribute is the foreign key of **Comment(ID).**
  - + reported_song: the reported song (= None if this is a comment-report). This attribute is the foreign key of **Song(ID)**.

4.4.10 PremiumPackage



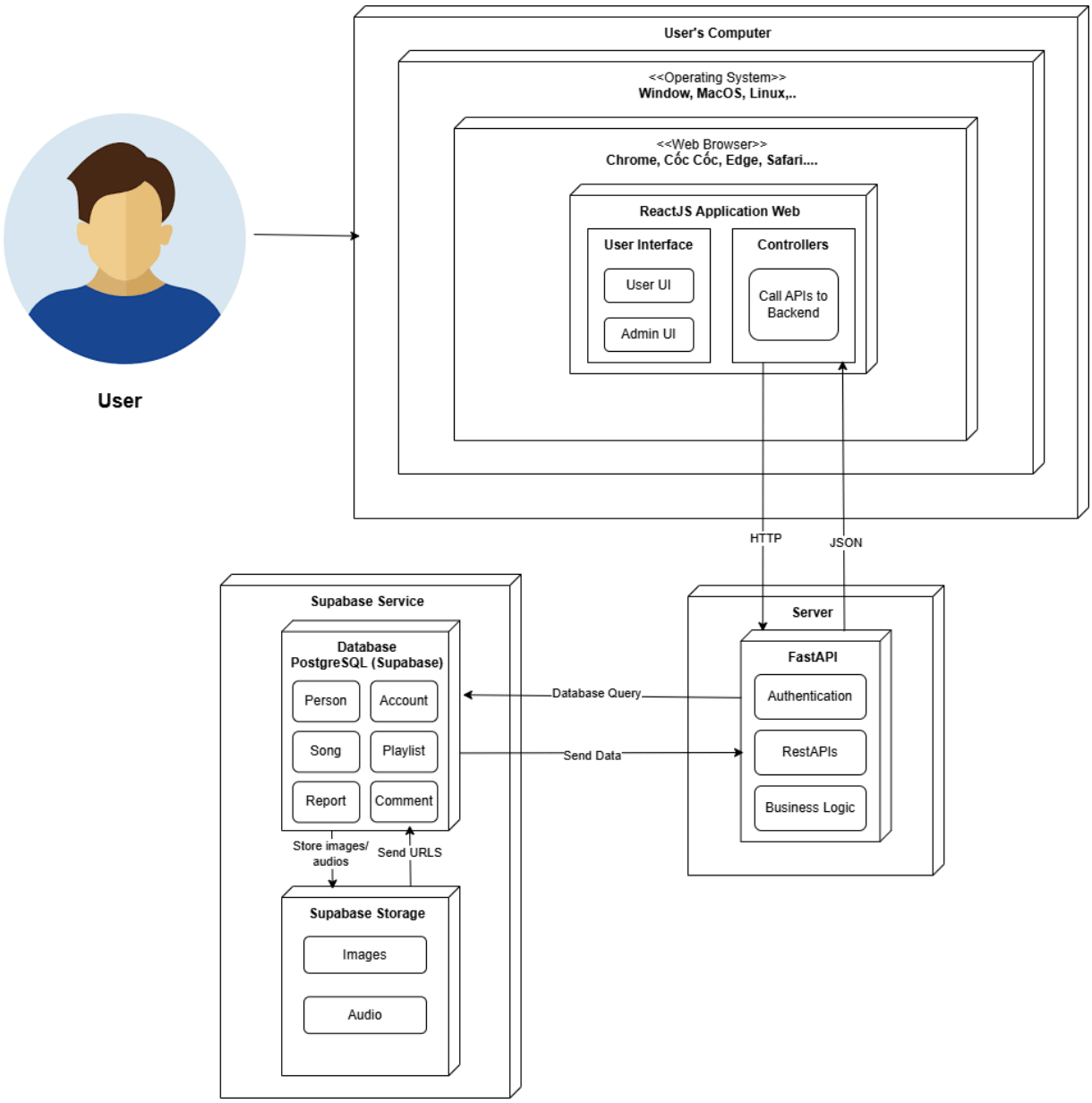
- This model stores information of premium packages, which users can pay for to upgrade their account to Premium account for a specific period of time.
- Attributes:
  + ID: model's primary key.
  + name: the package's name.
  + price: the package's price.
  + duration: number of days that user's account can maintain the Premium status after activating the package.

4.4.11 Subscriptions



- This **Subscriptions** model stores information about users' premium package subscriptions.
- Attributes:
  + ID: the model's primary key.
  + package_id: the package to which the user subscribes. This attribute is the foreign key of **PremiumPackage(ID)**.
  + user_id: the account of the user who subscribes to the package.This attribute is the foreign key of **Account(ID)**.
  + start_date: The date when the user subscribed to the package, and the premium account is activated.
  + expired_date: the date when the premium status expires.

5. Deployment



5.1 Frontend

- Technology: ReactJS.
- Deployed on: Web Server
- Device Supported: Desktop, Computer, Laptop
- Platform: Web browsers (Chrome, Edge, Safari, Cốc Cốc,..).
- Interaction: Users access website Melody Dive through a browser, interact with the user interface to view HomePage, ProfilePage, PlaylistPage, SongPage,....

5.2 Backend

- Technology: FastAPI.
- Deployed on: Cloud Server.
- Interaction:
    - Manipulate RestAPIs from Frontend.
    - Connect to Database (PostgreSQL on Supabase) to query data.
    - Manipulate business logic, authentication.

5.3 Database

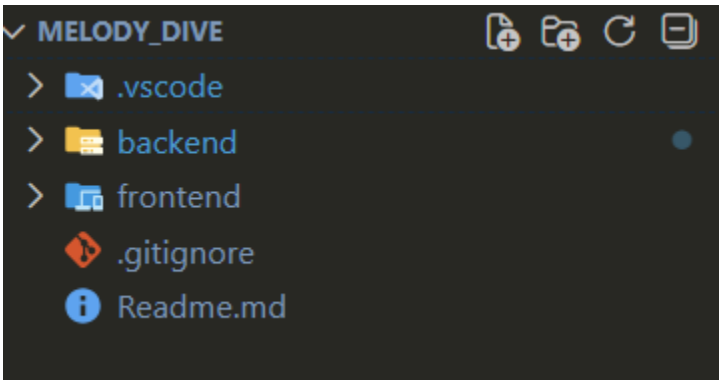
- Technology: PostgreSQL.
- Deployed on: Cloud database service. In our project, we use Supabase for storing metadata.
- Interaction: Send data for backend.

5.4 Supabase Storage

- Technology: Supabase Storage
- Deployed on: Cloud service. In our project, we use it to store images for the user, album, song, playlist's avatar and song's audio.
- Interaction: Send data for backend

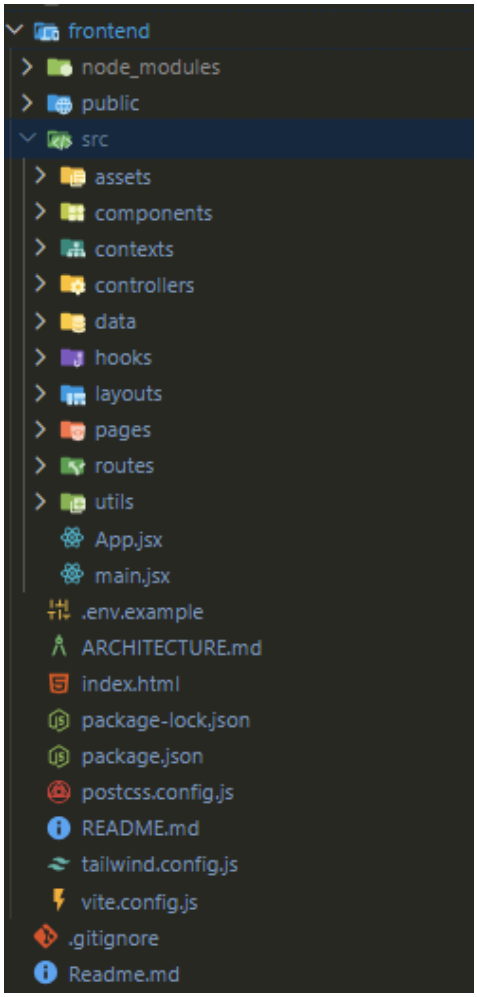
6. Implementation View

6.1 Folder Structure Overview



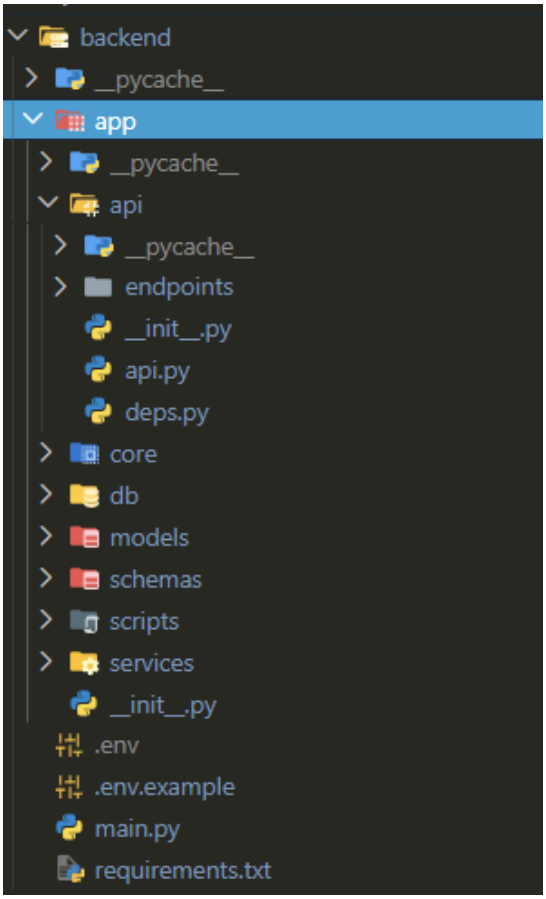
Melody_Dive/: Root project
- .vscode/: Configuration of IDE.
- backend/: Service Handle (Server)
- frontend/: Client Application (Client)
- Readme.md: Brief Introduction about our projects. Detailed information about folder structure.

6.2 Frontend



- public/: Contains static files served directly by the server (index.html, icons).
- src/assets/: Stores static resources such as global CSS stylesheets (css/index.css) and images.
- src/components/: Reusable UI building blocks organized by functionality (auth, common, features, ui, etc.).
- src/contexts/: Manages global application state using React Context API (AuthContext, MusicPlayerContext, UserContext).
- src/controllers/: Encapsulates business logic and API service calls (authController, musicController) to separate logic from UI components.
- src/data: Holds static and mock data files (mockData.js) used for development, testing, or initialization.
- src/layouts: Defines the high-level structural templates and wrappers for different page types (MainLayout, AuthLayout, AdminLayout).
- src/pages: Contains the main view components corresponding to specific application routes (admin, auth, user, guest).
- src/utils: Contains helper functions and utility scripts for common tasks shared across the application.
- App.jsx: The root component of the application, serving as the main container that sets up the overall structure and routing logic.
- main.jsx: The JavaScript entry point that initializes the React application, mounts the root component to the DOM, and often imports global styles.
- Package.json: Declares frontend dependencies (React, React DOM, ESLint, etc.).
- Package-lock.json: Records the exact version of all dependencies to ensure consistent installation across environments.

6.3 Backend



- app/api/: Contains the API route definitions (endpoints) and dependency injection logic.
    - endpoints/: Holds the specific route handlers grouped by resource (e.g., users, songs, auth).
    - deps.py: Defines reusable dependencies (like current user injection).
- app/core/: Stores core application configuration settings (config.py), security logic, and constants.
- app/db/: Manages database connections and session handling
- app/models/: Defines the database models (ORM classes) representing tables in the database (SQLModel/SQLAlchemy).
- app/schemas/: Contains Pydantic models (Data Transfer Objects) used for request validation and response serialization.
- app/scripts/: Holds standalone scripts for tasks like database initialization or data migration.
- app/services/: Encapsulates business logic, separating it from the API layer (e.g., handling complex operations like file uploads or music processing).
- main.py: The entry point of the application where the FastAPI app is initialized and routers are included.
- requirements.txt: Lists all Python dependencies required to run the project.